PeakMeister: Original Code Sections

Section descriptions from the original GitHub version

This document reproduces the section list from server section 1 through the end of the UI section list. It describes the original master code, before the modular-server changes. Numbering is preserved and spelling is lightly corrected. The section comments mark code blocks; they were not separate functions in the original version.

Source: AmirMazouchi/PeakMeister2.0_UI, original master
Commit: d7835b0b8a0868e8c786ae7a9a817f972dc740dd

1. About tab

Displays the README, disclaimer, update history and license. It reads the documentation files and places their contents in the About page.

2. User supplied inputs tab

Manages the information supplied before analysis: metabolites, reference compounds, processing parameters, project information and migration indexes.

Section	Section comment	What it does
2.1	**Initialize environment**	Creates the reactive tables that hold the mass list, reference list, parameters and user-supplied migration indexes. Connects these tables to their GUI displays.
2.2	**Data Table Input**	Reads the uploaded Excel workbook's Mass List, Reference Mass List and Parameters sheets. Stores their contents for subsequent analysis.
2.3	**Manual Mass List Input**	Handles adding metabolite entries manually, deleting selected rows and clearing the mass list. Entries include m/z, extraction settings, filtering settings and expected migration times.
2.4	**Manual Reference Mass Input**	Handles manual entry and removal of reference compounds. Stores their extraction settings, acceptable migration-time ranges and expected injection peak times.
2.5	**Manual Parameters Input**	Handles adding, deleting and clearing general processing parameters. These include injection count, lock masses, calibration, smoothing and manual-index settings.
2.6	**Project information input**	Stores and displays the project name, description, supervisor and contact information. Also handles clearing those fields.
2.7	**Manual MTI input**	Loads the user-supplied migration-index workbook or accepts manually entered rows. Handles deletion and clearing of these entries.

Section	Section comment	What it does
2.8	**Save input information**	Downloads the current mass list, reference list and parameters as an Excel workbook. This lets users retain their configured inputs.

3. Engine tab

Contains the main analysis pipeline. It accepts data files, prepares migration indexes, extracts electropherograms, detects and filters peaks, generates plots and saves results.

Section	Section comment	What it does
3.1	**mz5 file upload**	Manages the files or filenames available to the application. Includes both real-file uploads and the original pseudo-file workflow.
3.1.1	**Upload real .mz5 files**	Collects uploaded filenames and temporary file paths, displays the file list and handles clearing it. Can automatically trigger analysis after an upload.
3.1.2	**Upload file names and generate pseudo .mz5 files**	Accepts filenames from an editable table or Excel file. Generates empty .mz5 placeholders so saved results can be accessed without uploading the large original files.
3.2	**Initialize Run Button – Main Content**	Starts the main processing action. Creates a uniquely named results folder, checks for duplicate compound names, reads the injection count and saves the input workbook.
3.3	**Migration Index Calculation**	Calculates migration indexes from the expected migration times supplied in the workbook. Selects reference compounds and uses either a two-reference index or a single-reference time ratio.
3.4	**User Supplied MTIs**	When manual indexes are enabled, checks the supplied names and injection columns. Replaces the calculated entries for the specified metabolites and saves the migration-index summary.
3.5	**Data File Analysis**	Begins the loop over uploaded files and sets up progress reporting. Creates each file's data subfolder, makes a temporary raw-file copy and reads its MS1 spectra.
3.6	**Perform Mass Calibration**	Optionally corrects spectrum m/z values using two known lock masses. Spectra without both acceptable reference signals are left uncorrected.
3.6.1	**Develop correction model**	Fits a straight line relating measured m/z to the errors observed at the two lock masses. Calculates a separate model for each usable spectrum.

Section	Section comment	What it does
3.6.2	**Apply correction model**	Applies the calculated correction to the spectrum's m/z values. Uses linear correction between the references and constant corrections outside them.
3.7	**Extract Electropherograms**	Builds m/z extraction windows for the metabolites and internal standards, removes negative m/z values and sorts spectra where necessary. Extracts mean-intensity electropherograms into eie_df.
3.8	**Smooth Intensity Vectors**	Optionally smooths the extracted intensity traces using the specified kernel and smoothing strength. The resulting traces feed the peak-detection stages.
3.9	**Internal Standard Peak Detection, Integration, and Filtering**	Finds and selects internal-standard peaks for the expected injections. Their detected migration times become landmarks for locating metabolite peaks.
3.9.1	**Peak detection**	Identifies rising and falling portions of each internal-standard trace. Records candidate peak starts, apexes, ends and corresponding intensities.
3.9.2	**Migration time filtering**	Removes internal-standard candidates outside the configured minimum and maximum migration times. This restricts processing to the expected part of the run.
3.9.3	**Integrate peaks**	Calculates candidate peak areas using trapezoidal integration. Subtracts a rectangular baseline based on the lower boundary intensity.
3.9.4	**FWHM filtering**	Calculates full width at half maximum and rejects unusually broad peaks using the configured tolerance. Then retains candidates with the largest areas for the expected injection count.
3.9.5	**Peak space filtering**	Checks whether successive internal-standard peaks have reasonably consistent migration-time spacing. Uses the median spacing and configured percentage tolerance to identify suspicious gaps.
3.9.6	**Scenario 1**	Handles the case where one abnormal gap is detected. Attempts to restore a missing peak from the candidate list, or creates a zero-area placeholder, while removing a suspected false assignment.
3.9.7	**Scenario 2**	Handles cases with two or three abnormal gaps. Tries removing a suspect peak and recovering the missing injection peak from the remaining candidates.
3.10	**Metabolite Peak Detection, Integration, and Filtering**	Detects metabolite peaks and assigns them to injection positions. Uses the detected internal standards and previously prepared migration indexes to guide selection.

Section	Section comment	What it does
3.10.1	**Peak detection**	Finds candidate metabolite peaks from changes in trace direction. Records peak boundaries, apexes and intensities.
3.10.2	**Filter peaks by peak width**	Rejects candidates narrower than the configured minimum peak width. This helps remove short signal fluctuations.
3.10.3	**Integrate peaks**	Calculates trapezoidal areas for the remaining metabolite candidates. Applies the same boundary-based baseline subtraction used for internal standards.
3.10.4	**Filter peaks**	Predicts each injection's metabolite migration time using its migration index and the detected references. Selects a nearby candidate within the configured window, or inserts a zero-area placeholder when none is found.
3.10.5	**Filter peaks outside of run time limits**	Handles predictions that fall beyond the recorded run duration. Inserts placeholders so the result retains an entry for each expected injection.
3.10.6	**Filter duplicated peaks**	Detects cases where the same candidate was assigned to multiple injections. Repeats selection with progressively narrower migration windows and uses placeholders if resolution fails.
3.10.7	**Filter using peak spaces**	Checks the spacing between assigned metabolite peaks against the expected injection pattern. Attempts to replace suspicious assignments using nearby accepted peaks as landmarks.
3.10.8	**Filter peaks below LOD**	Estimates a noise-based threshold from sections of the electropherogram. Marks qualifying results as <LOD; despite the section title, this stage primarily adds annotations.
3.10.9	**Filter interfered peaks**	Compares metabolite peak times with the times of compounds listed as potential interferences. Marks peaks as Interfered when they fall within the configured co-migration threshold.
3.11	**Plotting**	Prepares plot data and produces static and interactive electropherograms. Includes peak labels, integration regions and migration-reference information.
3.11.1	**Create annotation and peak fill data frame**	Builds the data used to label injection peaks and shade their integration regions. Packages the trace, annotations, integration data and axis settings into plot objects.
3.11.2	**Create plot function for ggplots**	Defines the common appearance of static electropherograms. Draws the trace, peak labels, shaded integration areas and axes.

Section	Section comment	What it does
3.11.3	**Create Plot Function for making plotly plots**	Defines the interactive electropherogram format. Adds interactive elements, including the vertical integration-boundary lines.
3.11.4	**Initialize Saving Locations and Folders**	Creates the plot-output directories for the selected organization format. Separates analyte and internal-standard plots where applicable.
3.11.5	**Save plots as ggplots**	Generates and saves static plot images. Adds reference-related information and organizes files according to the chosen plotting format.
3.11.6	**Save plots as editable plotly plots**	Builds the interactive plot objects and gives them names associated with their source file and compound. Stores them for display and later saving.
3.12	**Export Data**	Converts the processed peak information into result tables. Saves CSV reports, reusable R objects and database records.
3.12.1	**Generate peak area data frame**	Creates a table with one row per injection and columns for the compounds. Replaces numerical areas with annotations such as <LOD, NPD or Interfered where applicable.
3.12.2	**Generate peak migration time data frame**	Creates the peak-apex migration-time table in minutes. This block also saves the analysis snapshot, writes results to SQLite and removes the temporary raw-file copy.
3.12.3	**Export data**	Writes Metabolite Peak Areas.csv and Metabolite Migration Times.csv. Also saves the interactive plot collection as plotly_objects.RData.

Unnumbered comment within 3.10.8

Section	Section comment	What it does
—	**Annotate injections that are not detected**	Assigns NPD to injection positions with zero peak area. This distinguishes missing detections from the other annotations.

4. Visualization tab

Loads saved analyses and displays their electropherograms. It also provides peak deletion, integration changes, baseline adjustment, interference marking and regeneration of saved outputs.

Section	Section comment	What it does
4.1	**Define reactive expressions and specific required functions**	Establishes the shared state and data-access helpers used by the visualization tools. Includes saved-data loading and database operations.
4.1.1	**Define Reactive expressions**	Tracks the selected results folder, file, available plots and current plot data. Also tracks modified plots and triggers display refreshes.
4.1.2	**SQL Database functions**	Defines database connection, table creation, reading and writing helpers. Includes conversion of older RData results into SQLite and reconstruction of plotting and editing data.
4.2	**Environmental initialization: File upload, and folder selection**	Populates file and results selectors and opens the selected database. Offers database creation for older results and loads the selected analysis into the session.
4.3	**Render and display plots and annotation information**	Connects plot selection to the displayed electropherogram and peak-information table. Handles plot interaction state.
4.3.1	**Populate plot table and render selected plot**	Lists the plots associated with the selected file. Constructs the chosen plot from saved data and restores the current zoom where available.
4.3.2	**Functions for dealing with moveable lines on plotly plots**	Handles switching between zooming and drawing integration boundaries. Tracks zoom limits and the red and blue vertical-line positions.
4.3.3	**Render the annotation data table associated with selected plot**	Retrieves the selected compound's peak annotations from SQLite. Displays injection numbers, comments, apex times and peak heights in a selectable table.
4.4	**Delete Peaks and Update Metadata**	Groups the peak-deletion operations and their data updates. Here, "metadata" includes the app's peak-analysis structures, rather than only the sample-identification CSV.
4.4.1	**Initialize environment, define reactive expressions, and define functions**	Defines helpers for loading, saving and refreshing edited peak information. Maintains current analysis state and a previous-state snapshot for the original Undo mechanism.
4.4.2	**Server logic for deleting peaks in plots**	Deletes the peaks selected in the peak-information table. Updates annotations and integration data, saves the changes and refreshes the display.
4.4.3	**Button for deleting all but the first and last peaks**	Removes the interior injection peaks from the selected electropherogram. Keeps the first and last peak positions and updates the associated saved data.
4.4.4	**Button for deleting ALL peaks in a plot**	Applies deletion to every injection peak in the selected electropherogram. Updates the annotations and removes their integration regions.

Section	Section comment	What it does
4.5	**Undo peak deletion**	Restores the single previous state stored in memory and writes it back to the analysis snapshot. This was the original one-step mechanism, before our persistent multi-step Undo/Redo changes.
4.6	**Regenerate plots**	Defines tools for rebuilding saved plots and result CSVs after editing. Includes plotting functions, export helpers and GUI actions.
4.6.1	**Defining Plotting Functions**	Defines the static and interactive plot builders used during regeneration. These reproduce plots from the edited analysis data.
4.6.2	**Defining functions for modifying saved peak area information**	Reads peak areas from SQLite and reshapes them into the report format. Rewrites Metabolite Peak Areas.csv.
4.6.3	**Function for regenerating plotly plots**	Defines a helper that rebuilds interactive plots from saved analysis snapshots. Saves the resulting plot collection using compressed RData storage.
4.6.4	**Function for regenerating ggplots plots and saving them to specified locations**	Recreates static plot images using the edited peak data. Uses the uploaded parameter tables to determine plot organization and reference information.
4.6.5	**Function for Regenerating Migration time file**	Reconstructs the migration-time report from database apex annotations. Writes Metabolite Migration Times.csv.
4.6.6	**Applying functions**	Connects regeneration and CSV-rebuilding helpers to the buttons in the additional-options dialog. Also manages loading messages and the table of modified plots.
4.7	**Adjusting integration and adding peaks**	Groups the manual integration and baseline-editing tools. Lets users correct automatically assigned peaks or supply a missing peak's integration region.
4.7.1	**Function for manual peak number input**	Opens a dialog asking which injection peak should be modified. Used when the user has not selected a peak-table row.
4.7.2	**Integration function**	Integrates the trace between the red and blue vertical lines. Determines the apex and updates the selected injection's peak data, annotations and saved state.
4.7.3	**Applying Integration Functions**	Connects the manual-integration button and peak-number confirmation dialog to the integration function. Refreshes the displayed table and peak-area report afterward.
4.7.4	**Logic for Adjusting Individual Integration baselines**	Defines the calculation for changing one peak's baseline. Uses a clicked plot position to recalculate its integration and update the associated data.

Section	Section comment	What it does
4.7.5	**Applying manual baseline adjustment on individual peaks**	Activates individual-baseline mode and captures the user's plot click. Applies the adjustment to the selected peak or requests a peak number.
4.7.6	**Logic for Adjusting baseline across all peaks in selected plot**	Defines how a common baseline is applied across the selected compound's peaks. Recalculates the corresponding integrations and areas.
4.7.7	**Applying baseline correction across all peaks in selected plot**	Displays the baseline-adjustment plot and captures the chosen baseline height. Calls the all-peaks adjustment function and handles closing the dialog.
4.8	**Server logic for dual EIE viewing**	Lets users select traces for comparison in vertically stacked plots with a shared time axis. Despite "dual" in the title, the code supports comparing two or three traces.
4.9	**Marking Interferences**	Marks selected peak-table entries as interfered and removes their integration shading. Saves the updated annotations and marks the plot as modified.

5. Downstream Processing

Works with the exported peak-area and migration-time tables. It covers normalization, missing values, sample identification, quality assessment and batch correction.

Section	Section comment	What it does
5.1	**Initializing Environment for loading peak area and migration time data**	Defines CSV-loading helpers and the reactive tables used downstream. Connects the results-folder, file-selection and load-data controls.
5.2	**Plot for m/z vs migration time**	Builds a scatterplot for the selected file and injection. Shows each compound's m/z and migration time, distinguishing metabolites from internal standards.
5.3	**Normalize to Creatine, F-Phe, or Cl-Tyr**	Divides each row's compound areas by the selected normalization compound's area. Updates the working table and writes a separate normalized CSV; the actual choices extend beyond those named in the heading.
5.4	**Replacing missing values in data**	Converts nonnumeric area annotations into missing values. Replaces them with either zero or one-fifth of the minimum available value for that compound, then saves a separate CSV.
5.5	**Connect Peak Position with Metadata**	Reads metadata.csv from the selected analysis's Data folder in the original workflow. Matches filenames and injection numbers to sample IDs, adds PBM Sample ID and saves the merged table.

Section	Section comment	What it does
5.6	**Control Charts**	Sets up monitoring plots for a selected reference compound. Compares its peak areas across the available sample rows.
5.6.1	**Control Charts for Non-normalized data**	Builds a chart directly from the original peak-area CSV. Displays the areas, their mean and limits at mean ± three standard deviations.
5.6.2	**Control Charts for normalized data**	Builds the corresponding chart from the current downstream working table. Therefore, it reflects whatever processing has been applied, even if normalization was skipped.
5.7	**CV and PCA determination for QC variance checks**	Assesses variability in QC and non-QC samples. Produces average CVs, compound-specific CV curves and a PCA plot.
5.7.1	**Define functions to calculate CV**	Defines the per-compound calculation SD / mean × 100. Applies it across the numerical compound columns.
5.7.2	**Define functions for creating PCA plots**	Defines data preparation, zero-variance filtering and scaling for PCA. The original implementation calls PCA() after preparing the matrix.
5.7.3	**Function for extracting QC and non-QC categories from user input**	Separates sample rows using the QC identifier and exclusion fields. Checks usable data, calculates group CVs and prepares combined data for PCA.
5.7.4	**Output for CV processing**	Runs the quality-assessment calculations when Calculate CVs is clicked. Connects their results to the displayed table and plots.
5.7.4.1	**CV plot**	Displays average QC and non-QC CVs in a table. Plots each compound's CV against m/z, with median CV reference lines.
5.7.4.2	**Create PCA plot**	Plots samples on PC1 and PC2, coloured by QC status. Adds group ellipses and explained-variance labels, and saves a PNG image.
5.8	**Display Data**	Displays the current downstream peak-area table in a scrollable table. This reflects changes such as replacement of missing values, normalization and metadata connection.
5.9	**Integrated Batch Correction**	Groups the package initialization and batch-correction workflow. Uses the working peak-area table together with an uploaded batch-order file.
5.9.1	**Initialize Environment**	Installs missing batch-correction packages and loads them. Also defines loading of the batch-order CSV.
5.9.2	**Perform Batch Correction**	Log-transforms compound areas and calls the correction routine using batch, sequence and reference-sample information. Runs reference-based and sample-based strategies, reverses the log transformation and exports results.

6. Reporting

Prepares an analysis matrix for external statistical analysis. Stores the generated matrix for display in the Reporting page.

Section	Section comment	What it does
6.1	**Generating Data Matrix for MetaboAnalyst**	Removes file and injection-position columns, transposes the peak-area table and adds sample IDs and a CLASS row. Assigns classes from the user's rules, displays the matrix and saves Metaboanalyst_matrix.csv.

7. Drug Analysis Tab

Contains the initial framework for loading adherence results and drug databases. The original section does not implement a complete drug-interaction or adherence-analysis pipeline.

Section	Section comment	What it does
7.1	**Load data and define reactive values**	Loads the adherence Excel file and local TWOSIDES and DrugBank databases. If a cached DrugBank RDS file is unavailable, offers to parse the local XML database and save a reusable cache.

The original ui.R section comments

ui.R has its own numbering. These sections define the visible controls and layout; the server sections above implement their behaviour.

Section	Section comment	What it does
1	**Setup**	Loads the required packages and defines the application title and dashboard shell. Missing packages can be installed during this step.
2	**Sidebar Main Tabs**	Defines the navigation entries for About, User Supplied Parameters, Engine, Visualization, Downstream Processing, Reporting and Drug Analysis.
3	**About Main Tab Content**	Defines the About-page documentation panels and shared interface styling. Provides output locations populated by the server.
4	**User Supplied Inputs Main Tab Content**	Organizes the workbook inputs and manual-entry controls into subtabs.
4.1	**Mass List Subtab**	Defines metabolite workbook upload, the mass-list table and manual metabolite inputs. Includes expected migration-time fields and row-management buttons.
4.2	**Reference Mass List Subtab**	Defines the reference-compound table and manual reference inputs. Includes extraction settings, migration limits and expected peak times.
4.3	**Parameters list subtab**	Defines general analysis settings such as injection count, calibration, smoothing and plotting format. Includes the parameter table and row-management controls.
4.4	**Project Information Subtab**	Defines the project-description fields and the controls for saving or clearing them.
4.5	**Manual Migration Index Input**	Defines the migration-index workbook upload, manual index-entry fields and associated table controls.
5	**Engine**	Defines real-file upload, automatic-start settings, the original pseudo-file controls and Initialize Run. Includes locations for the uploaded-file table and progress display.
6	**Visualization tab**	Defines saved-data selection, plot and peak tables, editing buttons and trace-comparison panels. Provides the interface for reviewing and correcting peaks.
7	**Downstream processing tab**	Defines results loading, normalization, missing-value handling, metadata connection and control-chart controls. Includes the m/z–migration-time, quality, metadata and batch-correction panels.

Section	Section comment	What it does
8	**Reporting Tab**	Defines sample-classification rules, the MetaboAnalyst matrix-generation button and its output table.
9	**Drug Analysis Tab**	Defines adherence-result upload and the button for loading drug databases. Includes a note that users must obtain those databases separately.